

모델 구현 가이드 (LSTM / Transformer Encoder)

데이터 전처리·파이프라인(`data_pipeline.py`)을 쓰는 모델 팀원용 가이드입니다.

모델 코드는 `models.py`의 `LSTMClassifier` / `TransformerEncoderClassifier` 스텝을 채우면 됩니다. 파이프라인 인터페이스는 고정되어 있으니 아래 파이프라인만 지키면 두 모델이 그대로 붙습니다.

0. 환경 준비 (conda, Python 3.11)

window기준 setting

```
# 최초 1회: Python 3.11 conda 환경 생성 + 의존성 설치
powershell -ExecutionPolicy Bypass -File setup_env.ps1
# (또는) conda env create -f environment.yml

# 이후 매번
conda activate agnews-dl
```

공유는 `.venv/conda` 폴더가 아니라 아래 파일들로 합니다. 각자 `setup_env.ps1`을 실행해 동일 환경을 만듭니다.

파이프라인 동작 확인: `python data_pipeline.py`

1. Quickstart (파이프라인 smoke test)

구현 전 스텝이 아니라 `AverageEmbeddingClassifier`(완성된 baseline)로 파이프라인이 batch→model→loss까지 연결되는지 먼저 확인합니다.

```
import torch
import torch.nn as nn
from data_pipeline import DataConfig, build_pipeline, set_seed, to_device
from models import AverageEmbeddingClassifier

set_seed(42)

cfg = DataConfig() # max_len=128, batch_size=64, vocab 20k, seed=42
bundle = build_pipeline(cfg)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = AverageEmbeddingClassifier(
    vocab_size=bundle.vocab_size,
    num_classes=bundle.num_classes,
    pad_idx=bundle.pad_idx,
```

```

        embed_dim=128,
    ).to(device)

    batch = to_device(next(iter(bundle.train_loader)), device)
    logits = model(batch["input_ids"], batch["lengths"])      # (B, 4)
    loss = nn.CrossEntropyLoss()(logits, batch["labels"])

    print(logits.shape)    # expected: (B, 4)
    print(loss.item())

```

이 quickstart는 파이프라인 검증용입니다. 실제 main comparison은 `LSTMClassifier`와 `TransformerEncoderClassifier`로 수행

2. 배치 계약 (모든 DataLoader가 내놓는 dict)

키	타입 / shape	설명
<code>input_ids</code>	<code>LongTensor (B, L)</code>	<code>pad_idx</code> 로 패딩. <code>L</code> 은 배치 내 최대 길이($\leq \text{max_len}$)
<code>lengths</code>	<code>LongTensor (B,)</code>	행별 실제 토큰 수 (패딩 제외, 항상 > 0)
<code>labels</code>	<code>LongTensor (B,)</code>	클래스 인덱스 <code>0..3</code>
<code>texts</code>	<code>List[str] (len B)</code>	원문 텍스트 (실패분석용)
<code>indices</code>	<code>List[int] (len B)</code>	원본 dataset index
<code>orig_lengths</code>	<code>LongTensor (B,)</code>	truncation 전 토큰 수
<code>truncated</code>	<code>BoolTensor (B,)</code>	잘렸으면 True

추가 규칙:

- `input_ids.dtype == torch.long, labels.dtype == torch.long, lengths.min() > 0` (파이프라인이 assert).
- **패딩 마스크:** `pad_mask = (input_ids == bundle.pad_idx)` (True=패딩). 모델에서 생성.
- **LSTM:** `pack_padded_sequence(embeds, lengths.cpu(), batch_first=True, enforce_sorted=False)`.
- **Transformer:** `nn.TransformerEncoder(..., src_key_padding_mask=pad_mask)`.
- **mean pooling** 시 패딩 토큰을 평균에 포함하지 않기 (`AverageEmbeddingClassifier` 구현 참고).
- forward는 **logits (B, 4)** 반환. **softmax** 금지 (`CrossEntropyLoss`가 logits를 받음). softmax는 confidence/추론 분석에서만.
- `texts/indices`는 list라 GPU로 옮기지 않습니다. `to_device(batch, device)`가 텐서만 이동시킵니다.

3. PipelineBundle 필드

필드	용도
----	----

필드	용도
<code>train_loader / val_loader / test_loader</code>	<code>DataLoader</code> . <code>test_loader</code> 는 최종 평가 전용
<code>train_dataset / val_dataset / test_dataset</code>	<code>Dataset</code> 객체 (클래스 분포·실패분석 시 직접 접근)
<code>vocab</code>	<code>torchtext Vocab</code> . <code>vocab.get_itos()</code> 로 index→token 디코딩
<code>tokenizer</code>	<code>basic_english</code> 토큰나이저
<code>pad_idx / unk_idx</code>	0 / 1
<code>vocab_size</code>	실제 빌드된 vocab 크기 ($\leq 20,000$). <code>nn.Embedding</code> 에 사용
<code>num_classes</code>	4
<code>class_names</code>	<code>['World', 'Sports', 'Business', 'Sci/Tech']</code>
<code>label_to_name / name_to_label</code>	라벨↔이름 매핑 (혼동행렬 라벨·보고서용)
<code>config</code>	사용된 <code>DataConfig</code>

`DataConfig` 주요 필드: `max_len=128`, `vocab_size=20000`, `batch_size=64`, `seed=42`, `train_fraction=1.0`, `pad_to_max_len=False`(기본: 배치별 동적 패딩, True면 항상 128로 패딩), `return_text=True`, `return_metadata=True`.

4. 기준(base) 하이퍼파라미터 — 계획서

공유 학습 설정: **Adam**, **lr 1e-3**, **batch 64**, **8 epochs**, **dropout 0.3**, **seed 고정**, 사전학습 모델 금지.

	LSTM	Transformer Encoder
Embedding	dim 128	dim 128 + positional encoding
Core	2-layer, 단방향, hidden 256	2 layers, 4 heads, FF 256
Pooling	마지막 hidden state	non-pad 토큰 mean pooling
Head/Loss	linear → cross-entropy	linear → cross-entropy

5. 재현성 (동일 batch order)

`set_seed(seed)`와 `DataLoader` generator로 재현성을 확보합니다. 단, **두 모델이 완전히 동일한 shuffle order**를 보게 하려면 각 모델 학습 직전에 `set_seed(42)`를 다시 호출하고 `build_pipeline(cfg)`로 `DataLoader`를 새로 만듭니다. (한 모델 학습에 먼저 쓴 `DataLoader`는 generator state가 진행되기 때문입니다.)

```

set_seed(42)
bundle_lstm = build_pipeline(cfg)
# ... LSTM 학습 ...

set_seed(42)
bundle_tr = build_pipeline(cfg)
# ... Transformer 학습 ...

```

과제의 핵심 통제 조건은 동일 **split, tokenizer, vocabulary, max_len, metric, training budget**입니다.

6. Ablation 분담

바꾸는 것	어디서	누가
Embedding dim (64/128/256) — 필수 ablation	모델의 <code>embed_dim</code> 인자 (두 모델 동일하게)	모델 팀원
시퀀스 길이 (128 vs 256)	<code>DataConfig(max_len=...)</code>	데이터 파이프라인
데이터 크기 (25/50/100%) — 학습곡선	<code>DataConfig(train_fraction=...)</code>	데이터 파이프라인
Transformer depth(1 vs 3) / LSTM hidden(128/256/512) — 선택 확장	모델 인자	모델 팀원

7. 공고문 §11 Pitfalls — 모델·평가 측 체크리스트

데이터 측(라벨 0-3 assert, vocab 누수 방지, 패딩 정보 제공, truncation flag, seed, device 헬퍼, test 분리)은 파이프라인에서 처리했습니다. 모델/학습/평가에서 **반드시** 확인:

- ❑ **#6 train/eval 모드**: 학습 `model.train()`, 평가/검증 `model.eval() + torch.no_grad()`.
- ❑ **#7 dropout**: train/eval에서 동작이 다름. eval 모드를 빠뜨리면 검증 점수 왜곡.
- ❑ **#8 device**: `model, input_ids, labels` 동일 device. 배치는 `to_device(batch, device)`.
- ❑ **#9 metric**: 정확도 + **macro F1** — `from sklearn.metrics import f1_score; f1_score(y_true, y_pred, average='macro')`. 혼동행렬은 `class_names`로 라벨링.
- ❑ **#10 test set**: 모델 선택·튜닝·early stopping은 **val_loader**로만. **test_loader**는 맨 마지막 1회.
- ❑ **#3 padding**: LSTM pack / Transformer key_padding_mask + 마스킹된 mean.
- ❑ **#5 seed**: 모델 생성·학습 전 `set_seed(cfg.seed)` (§5 참고).
- ❑ positional encoding 적용(Transformer), 두 모델 **trainable 파라미터 수**를 `count_parameters`로 보고.
- ❑ forward output은 **logits** (softmax 아님).

8. 다음 단계(요약 워크플로)

1. `models.py`의 두 클래스 구현 → `python -c "import models"`로 import 확인.
 2. 학습 루프(공유): Adam/lr 1e-3/8 epoch, epoch마다 train-val loss 기록(손실곡선).
 3. 검증셋으로 best epoch 선택 → 마지막에 `test_loader`로 1회 평가.
 4. accuracy, macro F1, 손실곡선, 혼동행렬, 파라미터 수 표 생성.
 5. embedding dim ablation(64/128/256) 두 모델 모두.
 6. 실패분석: batch의 `texts/indices/truncated`로 오분류 예시(공유/LSTM-only/Transformer-only ≥5건) 분석.
 7. 최종 제출은 `train_eval.ipynb`(notebook) 중심으로 정리.
-